

Motivation Efficient VLA inference without retraining



The control loop of a frozen VLA policy (Fig. 2 of the paper). Green, orange, purple, and blue boxes mark where the five tricks enter; Redundancy Control is action repeat.

[1] VLA inference is slow. Tens to hundreds of milliseconds per step (60 to 450 ms in our runs), which limits how often the policy can observe, decide, and correct.

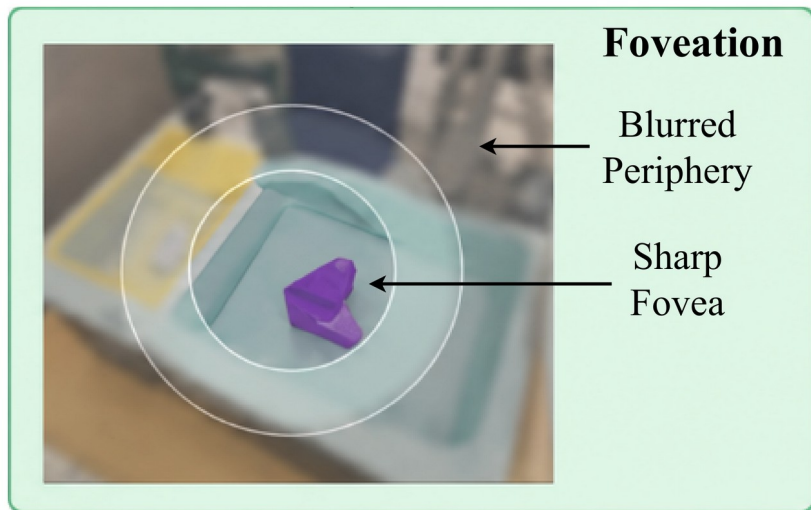
[2] Gains are reported in isolation. Training-free tricks are typically evaluated for a specific trick, backbone, benchmark, and implementation, so transfer across models is unknown.

Do the gains transfer across VLA models and environments, **or are they configuration-dependent?**

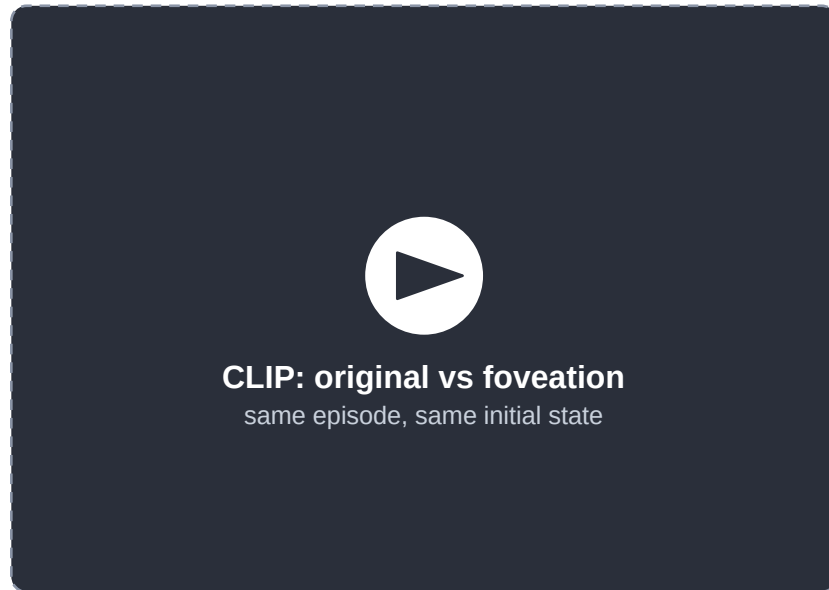
Vision-language-action models are capable but slow. Every policy call runs a vision encoder and a language backbone. Training-free tricks promise cheaper inference, but gains are typically reported for one backbone and one benchmark. Do they transfer?

Trick 1 Visual foveation

What to see



- Sharp central disc, progressively blurred periphery
- Keep ratio 20% or 50%; token count unchanged
- Tests whether high-frequency detail is needed only near the objects and the gripper

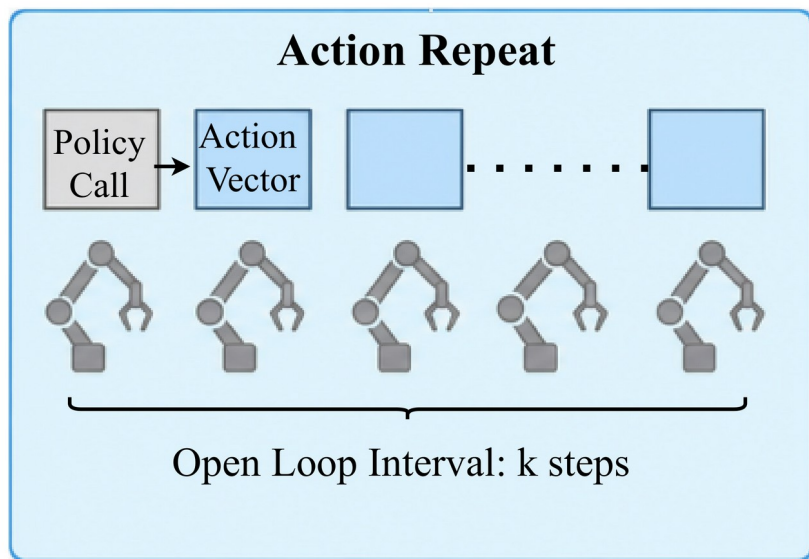


Rollout: original (left) and foveated observation (right), success or failure badge at the end.

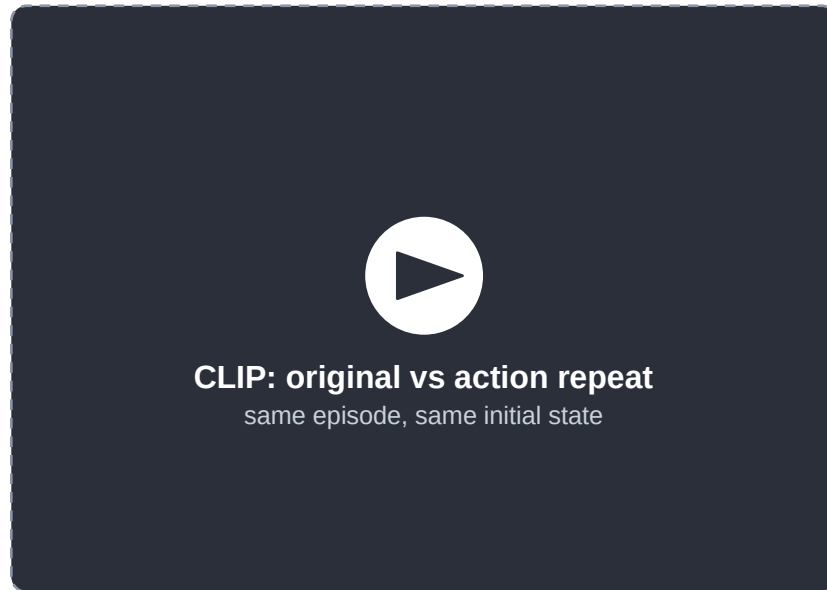
Trick one, visual foveation. The observation keeps a sharp central disc and blurs the periphery. Token count and compute are unchanged. Only what the policy sees changes.

Trick 2 Action repeat

When to act



- Hold each predicted action for $k = 2$ or 4 steps, about one call per k steps
- Cuts policy calls by about k , but lengthens the open-loop interval
- Fixed reduction, no state check: a speed-oriented baseline

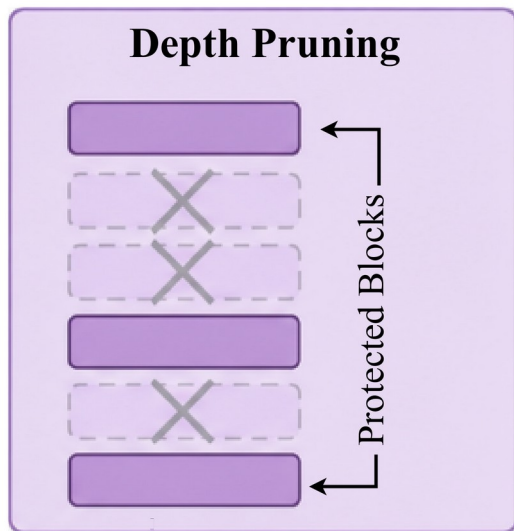


Rollout: watch contact and placement, where a held action can miss a needed correction.

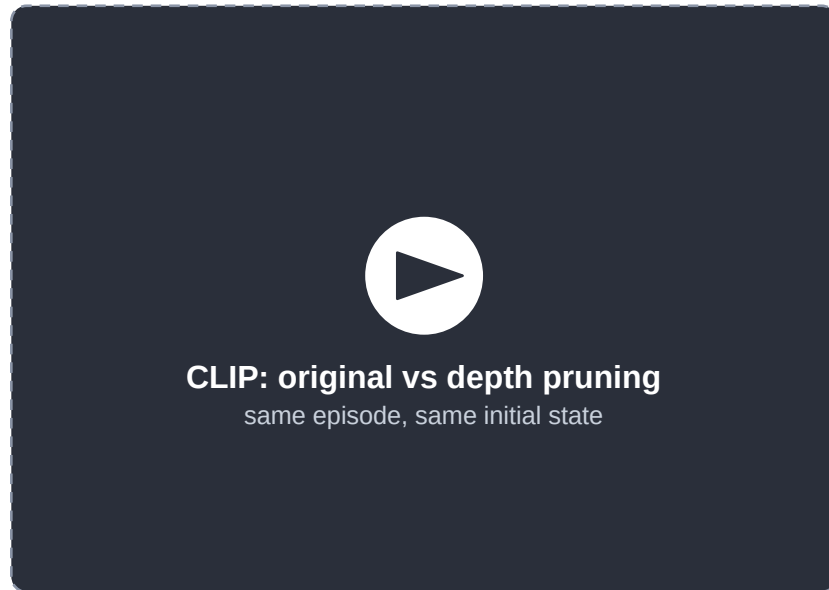
Trick two, action repeat. Each predicted action is held for two or four control steps, so the policy is called less often. No state check, a pure speed baseline.

Trick 3 Depth pruning

How much to compute



- Score each decoder block by Block Influence
- Remove 1, 2, or 4 low-influence blocks; none adjacent, early blocks and the final block kept
- Fixed once from calibration data, not adapted at test time

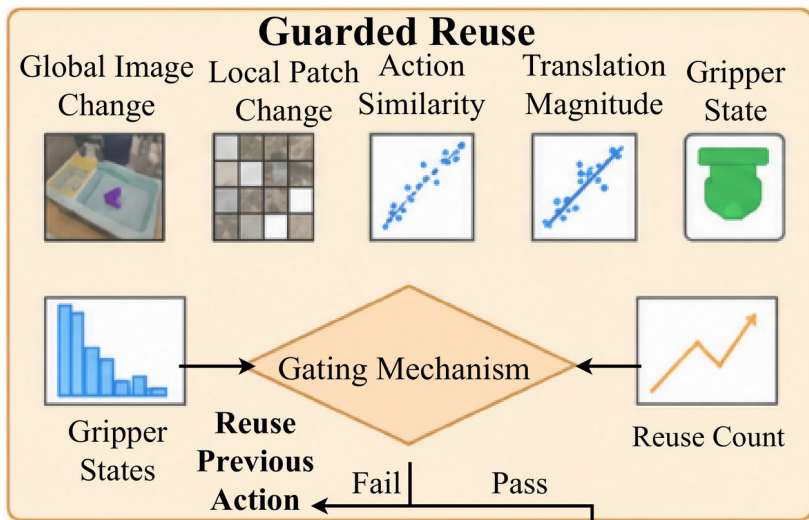


Rollout: CogACT on WidowX, where pruning removes decoder layers without breaking the closed loop.

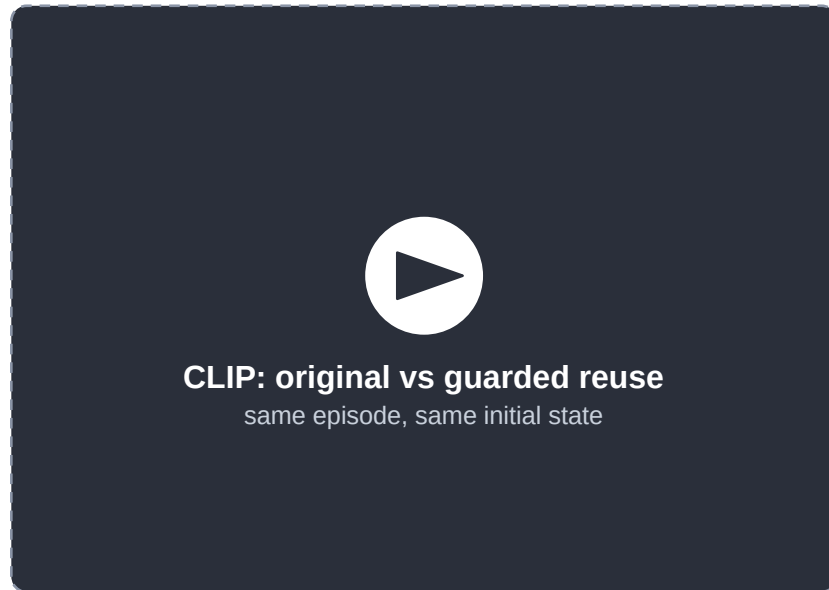
Trick three, depth pruning. Decoder blocks that barely change their hidden states are removed, one, two, or four of them, keeping the early and final blocks. The selection is fixed before evaluation.

Trick 4 Guarded action reuse

When to act



- Reuse the previous action only when all gates pass: small global and local image change, recent actions agree, translation not near zero, gripper unchanged, reuse count below a cap
- Presets: strict, moderate, aggressive

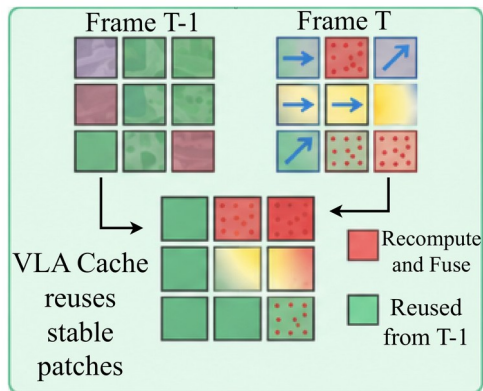


Rollout: overlay the gate indicators and a "calls skipped" counter on the right pane.

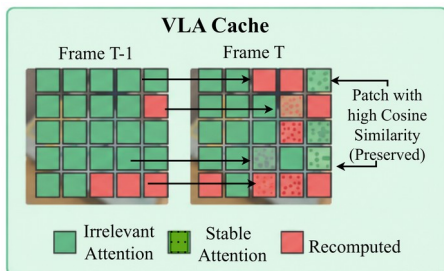
Trick four, guarded reuse. The previous action is reused only while every gate passes: image change, action agreement, translation, and gripper. Any failed gate restores a full call.

Trick 5 Temporal fusion

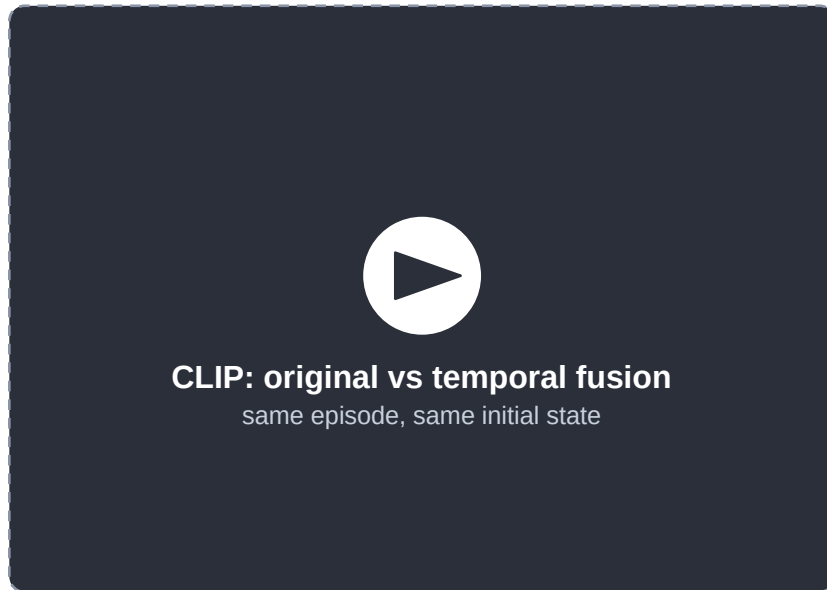
How much to compute



- Reuse visual tokens of stable patches from the previous call
- Recompute patches with motion, local structure, language attention, or in the interaction region
- Presets: motion-entropy, task-aware, conservative-adaptive



Top: fusion mask. Bottom: VLA-Cache criterion.



Rollout: optionally tint reused patches on the right pane to show what is carried over between frames.

Trick five, temporal fusion. Visual tokens of stable patches are reused from the previous call, while patches with motion, entropy, or language attention are recomputed. The policy still runs every step.

Results: matched evaluation protocol

Backbone	WidowX	Fractal	LIBERO
CogACT (7.6B)	•	•	
OpenVLA (7B)	•	•	•
SpatialVLA (4B)	•	•	
CronusVLA (0.5B)	•	•	
UniVLA (8.5B)	•		•
MiniVLA (1B)	•		
SmolVLA (0.45B)			•

SimplerEnv WidowX: 4 tasks × 50 episodes. Google Robot/Fractal: 5 tasks × 50. LIBERO: Long, Goal, Object, Spatial, each suite a separate pair (3 × 4 = 12), giving 22 pairs in total.

22

backbone ×
environment pairs

13 + 1

trick settings +
original, per pair

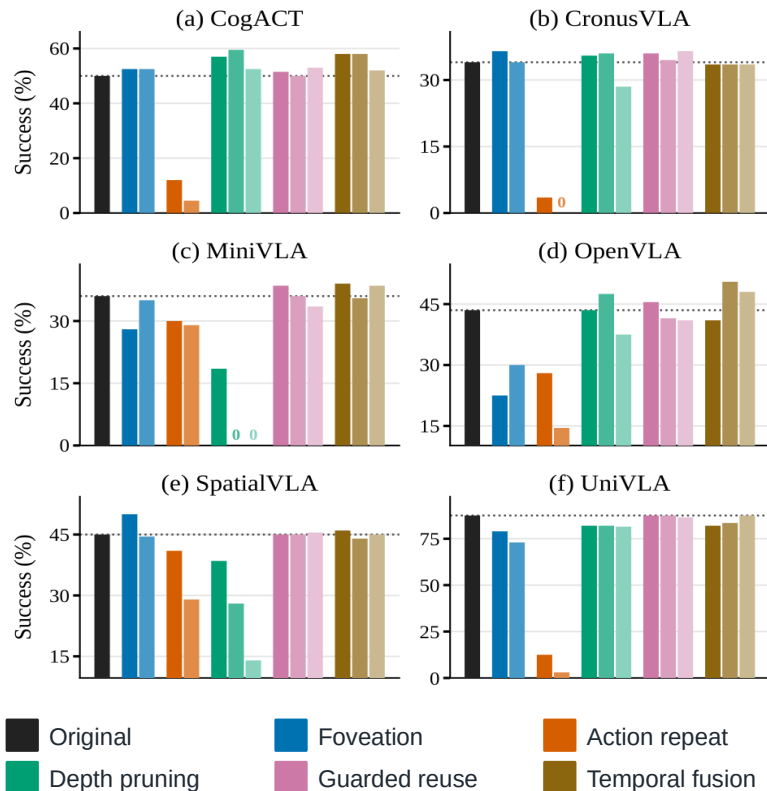
286

trick settings (13 ×
22), all paired with
the original

- Official checkpoints, no retraining
- Same task instances and initial states as the original
- Latency = wall-clock per environment step
- Exact two-sided McNemar test on paired episodes
- 4 × RTX 5090; tables report the best setting per trick

Seven backbones, official checkpoints, on WidowX, Fractal, and LIBERO. Thirteen trick settings per pair plus the original, 286 trick settings, each compared with the original on the same episodes.

Which tricks preserve success?



Q1 Success

Guarded reuse and temporal fusion are the safest for preserving success.

Selected backbones also gain:

Foveation, WidowX: CogACT +2.5 · SpatialVLA +5.0 · CronusVLA +2.5

Depth pruning, WidowX: CogACT +9.5

Backbone-dependent:

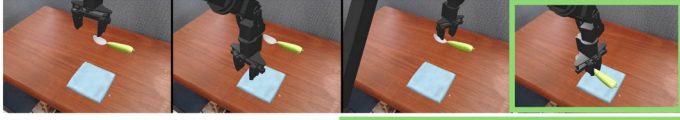
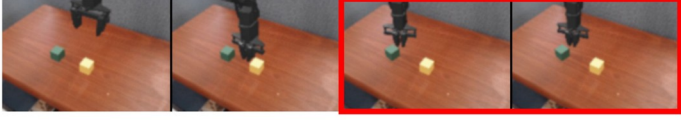
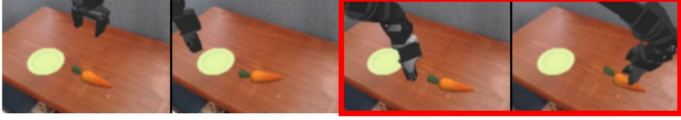
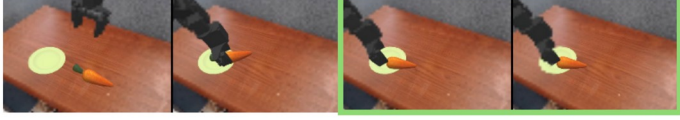
Foveation, LIBERO Goal: UniVLA +7.0 vs SmoIVLA -13.0

Action repeat: mainly a speed baseline

Fig. 1: success per configuration, six WidowX backbones (14 bars each, darker to lighter within a trick). Dotted line = original policy, 0 = no successful episodes. Deltas from Tables I and II, best setting per trick.

First, success. Guarded reuse and temporal fusion are the safest for preserving success. Foveation and depth pruning help selected backbones. Action repeat is mainly a speed baseline.

Depth pruning helps CogACT on WidowX

Instruction	CogACT Original	CogACT + Depth Pruning
Put the eggplant in the basket.		
Put the spoon on the table cloth.		
Stack the green cube on the yellow.		
Put the carrot on the plate.		

▶ CLIP: original vs depth pruning (paired episode)

Success 50.0 → 59.5 % **Latency 141.4 → 137.5 ms/step**

Red = failure, green = success. Selected episodes, same task instances and initial states. Aggregate values from Table I.

CogACT on WidowX. In these paired episodes the original fails and depth pruning succeeds. Overall success rises from 50.0 to 59.5, and latency drops slightly.

Same trick, same benchmark, different backbone



CLIP: MiniVLA original
succeeds (green border)



CLIP: MiniVLA + pruning
fails (red border)

Depth pruning, WidowX

CogACT +9.5

MiniVLA -17.5

SpatialVLA -6.5

Selected episode, same initial state, WidowX

Structural redundancy is not a property of the trick alone. It depends on how each backbone uses its decoder layers during control.

Same trick, same benchmark, different backbone. On MiniVLA, depth pruning drops success by 17.5 points, on SpatialVLA by 6.5. Here the pruned policy fails.

Which tricks are actually faster?

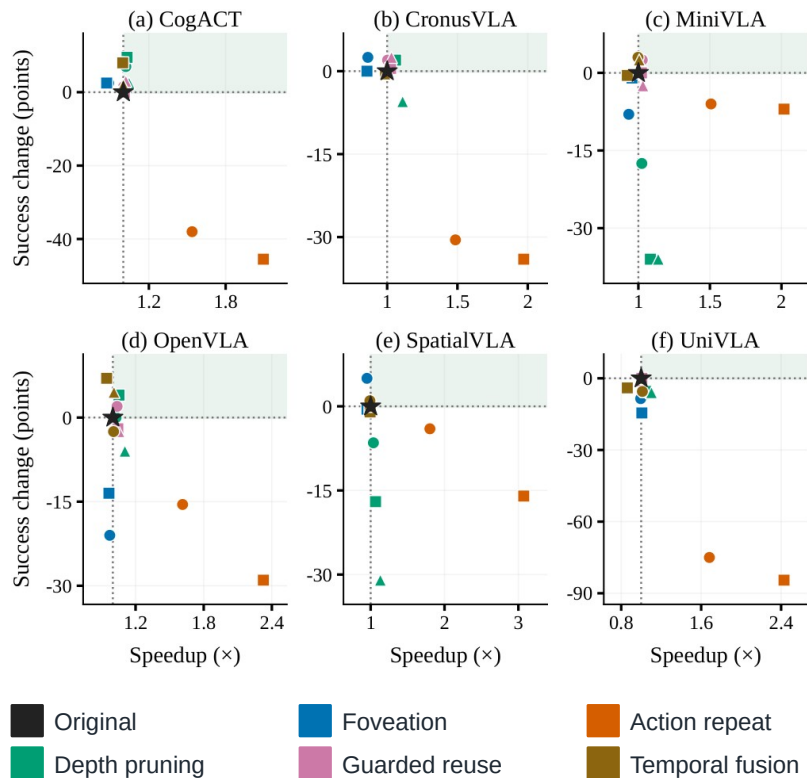


Fig. 4, SimplerEnv WidowX. Star = original, shaded = faster and no worse. Colour = trick; circle, square, triangle = first, second, third setting of that trick.

Q2 Speed

Action repeat gives the largest speedup, depth pruning gives decoder-level savings, guarded reuse gives the safest trade-off.

Latency, ms per step	Original → trick
Repeat · SpatialVLA WidowX	423.5 → 235.1
Repeat · OpenVLA LIBERO Long	171.6 → 93.2
Pruning · OpenVLA WidowX	210.2 → 199.4
Reuse · SmolVLA LIBERO Long	289.5 → 268.9

Second, speed. Action repeat gives the largest speedup. SpatialVLA on WidowX drops from about 424 to 235 milliseconds per step. Depth pruning gives smaller decoder-level savings, and guarded reuse the safest trade-off.

Open loop has a cost, gating keeps it safer



CLIP: Original

policy call every step

policy calls used: ____



CLIP: Action repeat

held actions, episode fails

policy calls used: ____



CLIP: Guarded reuse

calls skipped only when the gates pass

policy calls used: ____

Action repeat (k = 2, best setting), WidowX: CogACT -38.0 ·
UniVLA -75.0

Guarded reuse, SmoIVLA LIBERO Long: 289.5 → 268.9 ms

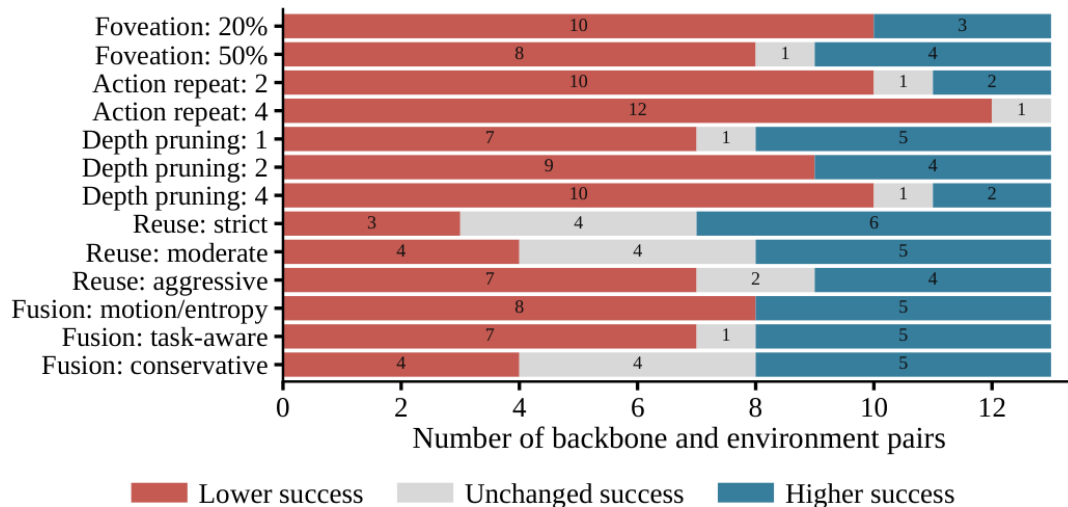
Fractal (milder): CogACT -1.2 · SpatialVLA 0.0

success unchanged at 42.0 %

Same CogACT WidowX episode and initial state in all three panels. Deltas from Tables I and II, best setting per trick.

Action repeat runs open loop: on WidowX, CogACT loses 38 points and UniVLA 75. Guarded reuse skips calls only when the gates agree. On SmoIVLA in LIBERO Long, success stays at 42.0 with lower latency.

Does the effect transfer across models?



For each setting: how many of 13 backbone and environment pairs got lower, unchanged, or higher success. LIBERO episodes are pooled over the four suites, so 13 pairs stand for the 22.

Q3 Transfer

Effects are a joint property of the trick, the backbone, and the task distribution.

Depth pruning, WidowX: CogACT +9.5 ·
MiniVLA -17.5

Foveation, LIBERO Goal: UniVLA +7.0 ·
SmoVLA -13.0

Action repeat: often harmful on WidowX,
milder on Fractal for some backbones

No trick is plug-and-play: each one needs matched evaluation under the target backbone and environment.

Third, consistency. Across backbone and environment pairs the same trick improves one model, leaves another unchanged, and hurts a third. No trick is plug-and-play.

Takeaways

Guarded reuse

Most reliable. Preserves success, modest speedup when the gates open.

Action repeat

Largest speedup, but the risk of long open-loop execution.

Depth pruning

Decoder-level savings, benefit depends on the backbone.

Foveation · Temporal fusion

Test how much the visual representation can change while the policy stays stable.

VLA efficiency is policy- and environment-dependent. Evaluate every trick under the target backbone and environment before applying it.

Guarded reuse is most reliable. Action repeat is fastest but risky. Depth pruning is backbone-dependent. Evaluate every trick under the target backbone and environment before applying it. Thank you.